Real-world (rebuild Project 2 as a graph) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `agent_loop_to_graph_practice.py`: this is where Doc07's whole ReAct loop gets rebuilt as a graph — the same "think, act, observe, repeat" shape, but now as a `think` node and an `act` node connected by edges instead of a `for` loop, with Doc07's own `AGENT_TOOLS` and `run_tool()` still doing the real work inside the `act` node. Seeing the exact same logic wear graph clothing is what makes a `StateGraph` feel like a structuring choice, not a different kind of agent. **If not:** the Build Task's graph would be the first place you ever turned a loop into a graph, with no smaller version to trust it against.

Solution

```
# agent_loop_to_graph_practice.py
import operator
from typing import Annotated, TypedDict

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

# Doc07's tools and tool-runner, inline so this file runs on its own.
@tool
def add(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b

AGENT_TOOLS = [add]

def run_tool(call: dict, tools: list):
    # find the tool by name and run it with the model's arguments
    for candidate in tools:
        if candidate.name == call["name"]:
            try:
                return candidate.invoke(call["args"]), None
            except Exception as error:
                return None, str(error)
```

```
    return None, f"unknown tool: {call['name']}"

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    steps: Annotated[list[dict], operator.add]

def think(state: AgentState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(AGENT_TOOLS).invoke(state["messages"])
    return {"messages": [response]}

def act(state: AgentState) -> dict:
    tool_messages: list[dict] = []
    steps: list[dict] = []
    for call in state["messages"][-1].tool_calls:
        result, error = run_tool(call, AGENT_TOOLS)
        observation = result
        if error is not None:
            observation = f"Error: {error}"
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({"tool": call["name"], "result": result, "error": error})
    return {"messages": tool_messages, "steps": steps}

def route(state: AgentState) -> str:
    if state["messages"][-1].tool_calls:
        return "act"
    return "finish"

def build_graph():
    builder = StateGraph(AgentState)
    builder.add_node("think", think)
    builder.add_node("act", act)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route, {"act": "act", "finish": END}
    )
    builder.add_edge("act", "think")
    return builder.compile()

def run_graph(graph, task: str) -> dict:
    start = {"messages": [{"role": "user", "content": task}], "steps": []}
    # why: the graph's version of Doc07's max_iterations
    return graph.invoke(start, config={"recursion_limit": 20})
```

```
# Build the graph and run it on a task that needs the add tool.
graph = build_graph()
result = run_graph(graph, "What is 12 + 30?")
print(f"answer: {result['messages'][-1].content}")
for step in result["steps"]:
    print(f"  step: {step['tool']} -> {step['result']}")
```

Expected output:

```
answer: 12 + 30 is 42.
  step: add -> 42
```